Laboratory 3

1. Adrian /Iustin

1. System Requirements: P2P Shopping Assistant

1.1. Problem Description

The system is an advanced shopping assistant application that transforms the individual shopping experience into a collaborative (Peer-to-Peer) one. The core functionality consists of generating a dynamic indoor map of products within supermarkets by collecting GPS and Wi-Fi RTT coordinates from users at the exact moment they mark an item as "completed" in their list. The system aggregates this crowdsourced data to provide optimal navigation routes between aisles and to suggest which nearby store contains the highest density of items from the user's current list.

1.2. System Actors

1. **Shopper (Authenticated User):** Creates and manages shopping lists, shares them with others, checks off items, and receives real-time indoor navigation guidance.
2. **Collaborator:** A secondary user who can add, edit, or mark items as complete on a shared list simultaneously (Multi-Agent synchronization).
3. **P2P Data Aggregator (Internal System):** The core engine that processes location data points from all users to update the global "product-to-shelf" map.
4. **GPS / Wi-Fi RTT Service (External Actor):** High-precision location services providing sub-1-meter accuracy for indoor positioning.
5. **Store Database (External Actor):** A third-party service providing general store information, addresses, and basic floor plans.

1.3. Use Case Scenarios (Brief)

- **Manage Shopping List:** User adds items, quantities, and optional metadata (brand, price).
- **Shared Shopping (Multi-Agent):** Multiple users sync on a single list; when one user checks an item, it updates for all in real-time.
- **Record Product Location:** The app automatically captures the precise location (lat/long/floor) when an item is marked "complete."
- **Generate Dynamic Map:** The system calculates the average coordinates for products based on aggregated P2P history.
- **Route Optimization (TSP):** Calculation of the shortest path through the store based on the user's current position and the known coordinates of items.
- **Store Recommendation:** Identifying the best shopping center based on the availability of items in the P2P location database.

Laboratory 3

---

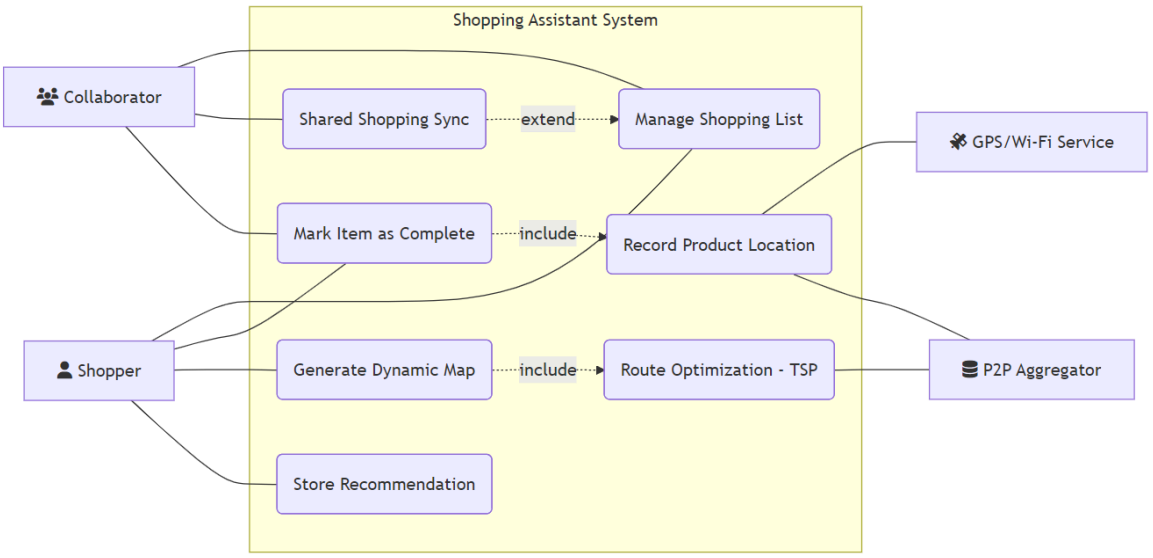
2. Class Diagram (Technical Specifications)

2.1. Main Classes and Members

- **Class `User`**:
  - *Attributes:* `userID: int`, `username: String`, `currentLocation: LocationData`.
  - *Methods:* `updateLocation()`, `shareList(listID)`, `markItemComplete(itemID)`.
- **Class `ShoppingList`**:
  - *Attributes:* `listID: int`, `name: String`, `isShared: boolean`.
  - *Methods:* `addItem(Item)`, `calculateRoute()`.
  - *Relation:* **Composition** with `Item` class.
- **Class `Item`**:
  - *Attributes:* `itemID: int`, `name: String`, `category: String`, `isCompleted: boolean`, `metadata: Map`.
- **Class `LocationData`**:
  - *Attributes:* `latitude: double`, `longitude: double`, `wifiRTT_Signal: double`, `accuracy: float`.
- **Class `P2PMapAggregator`**:
  - *Attributes:* `globalProductDatabase: Map<String, List<LocationData>>`.
  - *Methods:* `recordItemPickUp(itemID, location)`, `getAverageLocation(itemID)`.
  - *Relation:* **Aggregation** with `LocationData`.
- **Class `RouteOptimizer`**:
  - *Methods:* `solveMultiAgentTSP()`, `recalculateOnMove()`.

Laboratory 3

2. Bogdan



Link AI Tool Used: https://mermaid.live/

```
graph LR
    subgraph "SYSTEM ACTORS"
        U["fa:fa-user Shopper"]
        C["fa:fa-users Collaborator"]
        GPS["fa:fa-satellite GPS/Wi-Fi Service"]
        Aggregator["fa:fa-database P2P Aggregator"]
    end

    subgraph "USE CASE SCENARIOS (System Boundary)"
        UC1(Manage Shopping List)
        UC2(Shared Shopping Sync)
        UC3(Mark Item as Complete)
        UC4(Record Product Location)
        UC5(Generate Dynamic Map)
        UC6(Route Optimization - TSP)
        UC7(Store Recommendation)
    end

    U --- UC1
    U --- UC3
    U --- UC5
    U --- UC7

    C --- UC1
    C --- UC2
    C --- UC3
```

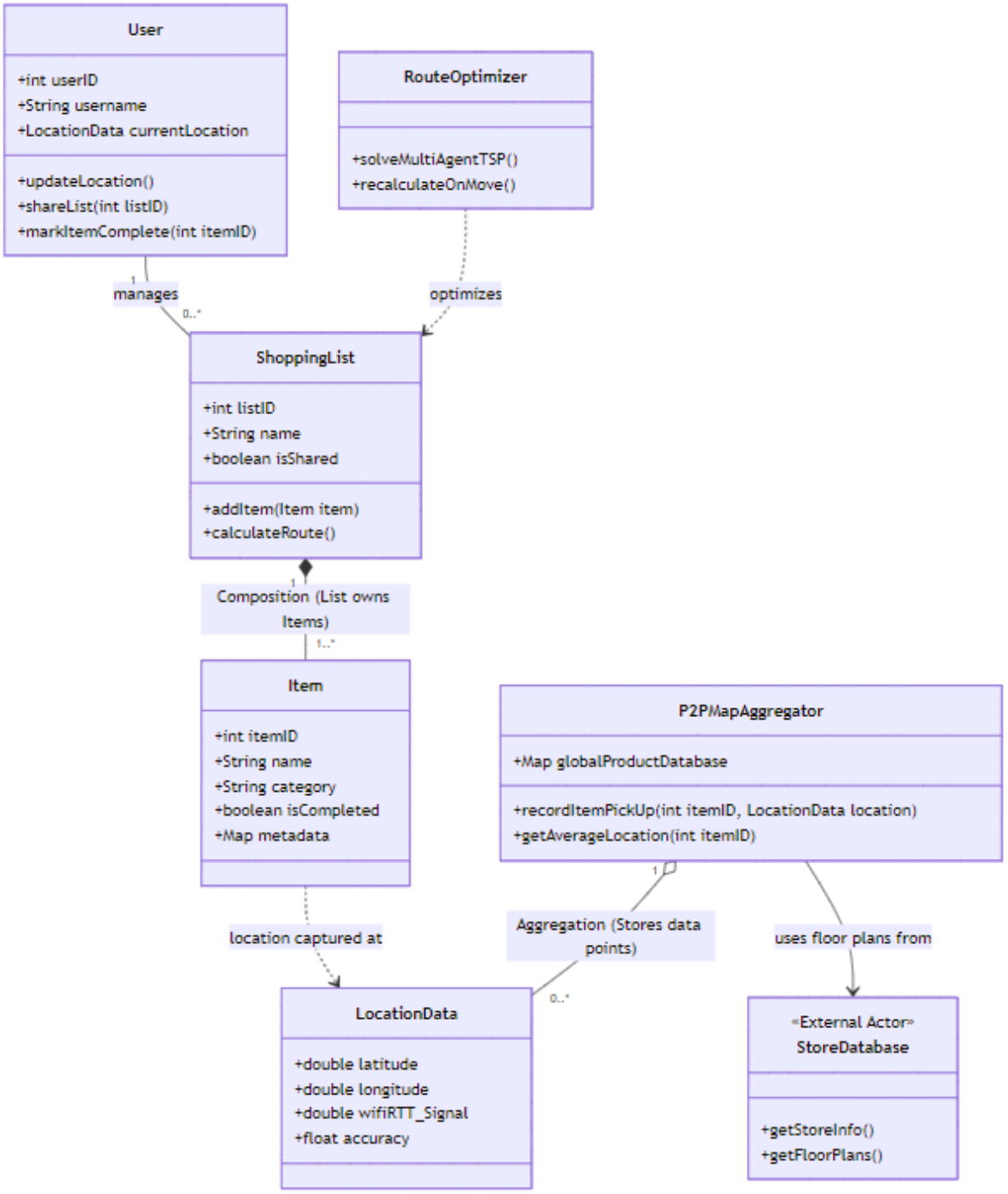
Laboratory 3

UC2 -.->|extend| UC1
UC3 -.->|include| UC4

UC4 --- GPS
UC4 --- Aggregator
UC5 --- Aggregator
UC6 --- Aggregator
UC7 --- Aggregator

Laboratory 3

3. Iustin



Link AI Tool Used: https://mermaid.live/

```
classDiagram
    class User {
        +int userID
        +String username
        +LocationData currentLocation
        +updateLocation()
        +shareList(int listID)
        +markItemComplete(int itemID)
```

```
}

class ShoppingList {
    +int listID
    +String name
    +boolean isShared
    +addItem(Item item)
    +calculateRoute()
}

class Item {
    +int itemID
    +String name
    +String category
    +boolean isCompleted
    +Map metadata
}

class LocationData {
    +double latitude
    +double longitude
    +double wifiRTT_Signal
    +float accuracy
}

class P2PMapAggregator {
    +Map globalProductDatabase
    +recordItemPickUp(int itemID, LocationData location)
    +getAverageLocation(int itemID)
}

class RouteOptimizer {
    +solveMultiAgentTSP()
    +recalculateOnMove()
}

class StoreDatabase {
    <<External Actor>>
    +getStoreInfo()
    +getFloorPlans()
}

%% Relații și Conexiuni conform cerințelor de laborator
User "1" -- "0..*" ShoppingList : manages
ShoppingList "1" *-- "1..*" Item : Composition (List owns Items)
P2PMapAggregator "1" o-- "0..*" LocationData : Aggregation (Stores data points)
Item ..> LocationData : location captured at
RouteOptimizer ..> ShoppingList : optimizes
P2PMapAggregator --> StoreDatabase : uses floor plans from
```

Advantages (Pros)

- **Efficiency:** Automated rendering from text is significantly faster than manual drag-and-drop tools.
- **Consistency:** Ensures perfect alignment and professional styling across all diagram types.

Laboratory 3

- **Version Control:** Being text-based, the diagrams can be easily stored and tracked in Git or Markdown documentation.
- **Instant Sync:** Changes in the technical requirements (attributes/methods) are reflected immediately in the visual model.

Disadvantages (Cons)

- **Strict Syntax:** A single typo (e.g., a missing bracket) leads to rendering errors like `UnknownDiagramError`.
- **Layout Control:** The algorithm decides element positioning; manual "pixel-perfect" adjustments are difficult.
- **Learning Curve:** Requires knowledge of specific Mermaid syntax compared to intuitive visual editors.